

Generic Trip Tracking Application — Product Specification

Version: 2.0 **Status:** Source of truth for implementation **Reference dataset:** Nagpur -> Agra -> Udaipur trip (used throughout as *example data only*) **Platform:** Native Android

How to read this document: every mention of Udaipur, Agra, Nagpur, City Palace, Lake Pichola, Jagdish Temple, or a specific hotel is sample data, not application behavior. Test: if a word like “Udaipur” could be deleted from the codebase without deleting a feature, the architecture is correct. If deleting it breaks something, that is a bug in the implementation, not this spec.

Contents

1. Product Purpose
2. Native Android Platform
3. Core Concept
4. Complete Trip Flow
5. Current Trip State
6. Home / Command Center
7. Timeline
8. Map and Location Tracking
9. Navigation
10. Train Journey Experience
11. Photo Inspiration and Planned Shots
12. Location Detail Page
13. Activity System
14. Automatic Progression
15. User Overrides
16. Plug-and-Play Trip Creation
17. Trip Editor
18. Data Model
19. External Services
20. Notifications
21. Offline Behavior
22. Photo and Memory System
23. End-of-Trip Experience
24. UI/UX Principles
25. Visual Design System
26. Edge Cases
27. Genericity Requirement
28. Expected User Experience Appendix A. MVP Scope & Build Order Appendix B. Final Architecture Principles

1. Product Purpose

The problem. A real trip is spread across train tickets, hotel confirmations, Google Maps locations, screenshots, messages, notes, and a manually written itinerary. The traveler still has to answer the same questions repeatedly: *Where am I? What am I doing now? Where do I go next? What do I need to do there?*

What the app does. The application turns a user-entered itinerary into a live trip companion. The user creates a trip by selecting locations from a map, adding timings, and attaching activities. During the trip, the application combines the stored itinerary with the current time and device location to show the most relevant state with minimal interaction.

How it should feel. It should feel like a polished visual travel HUD rather than a spreadsheet or project-management dashboard. The interface should be attractive, graphical, calm, glanceable, and usable one-handed while travelling. It should communicate information through hierarchy, progress indicators, route graphics, timelines, icons, and photography instead of filling the screen with text.

The second problem. Real trips deviate from plans. A train runs late, a visit runs long, a stop gets skipped, or the traveler decides to add something spontaneously. The application must remain useful when reality differs from the original itinerary. Manual overrides are therefore a core feature, not an emergency fallback.

What the app is not. It is not a booking engine, social travel network, full offline maps replacement, or AI-controlled travel planner. The current product intentionally contains **no LLM-dependent trip logic**. Trip progression, schedule calculations, GPS state, notifications, and user overrides remain deterministic and implemented in native application logic.

2. Native Android Platform

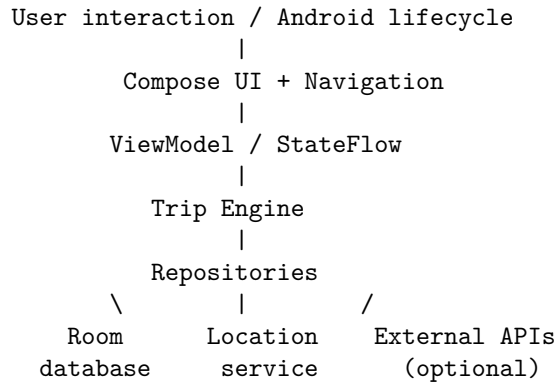
The application is a **native Android application**. Do not use Capacitor, a WebView wrapper, or a hybrid shell for the main product. The goal is to learn and control the Android platform directly, especially for location services, background behavior, notifications, local persistence, permissions, lifecycle handling, and performance.

Recommended foundation

- **Language:** Kotlin
- **UI:** Jetpack Compose
- **Navigation:** Navigation Compose
- **State:** ViewModel + Kotlin Coroutines + StateFlow
- **Local database:** Room

- **Preferences:** DataStore
- **Dependency injection:** Hilt
- **Background/deferred work:** WorkManager where appropriate
- **Location:** Android location APIs / Fused Location Provider
- **Maps:** a native Android map SDK/provider behind a small map abstraction

Native architecture flowchart



The Android layer is responsible for platform-specific concerns. The trip engine must remain independent of Android UI details wherever practical.

Tracking modes

The application has three user-facing modes:

1. **Planning Mode** - create and edit the trip; background tracking is normally off.
2. **Travel Mode** - active trip tracking, location updates, current/next state, navigation, and notifications.
3. **Review Mode** - browse completed events, photos, notes, and trip history after the trip.

The core trip data stays the same across all three modes.

3. Core Concept

The application is a **generic trip engine**: one data model and one set of screens that render *any* trip, driven entirely by data entered through a Trip Editor. There is no Udaipur-specific code path anywhere in the system — there is only Udaipur-specific *data*, created the same way any other trip's data would be created.

Entities

Entity	Definition	Key relationships
Trip	Top-level container for one travel plan (a date range, a purpose, a set of travelers)	Has many Travelers, many Events (ordered), one Settings object
Traveler	A person on the trip	Many-to-many with Trip (a person can appear on multiple trips)
Event	The atomic unit of the timeline — anything with a time and, usually, a place	Belongs to one Trip; references one or more Locations; may own Segments, Activities, Tasks, Media
Journey	An Event <i>subtype</i> representing transit between two Locations (train, flight, bus, car)	Composed of one or more Segments
Segment	One unbroken leg of a Journey — single vehicle, no change	Belongs to exactly one Journey
Location	A physical place with coordinates, address, and a category	Referenced by many Events; may carry a default Activity template and Photo Inspiration tags
Activity	A specific thing to do inside a Visit-type Event (explore, photograph, eat)	Belongs to one Event
Task	A generic actionable to-do, trip-level or tied to an Event (pack bags, buy tickets)	Optionally references one Event
Media	User-captured photo/note, or curated inspiration content	Optionally references an Event and/or a Location
Current Trip State	Not stored data — a computed answer to “what’s happening now,” recalculated from the entities above plus live time/GPS	Derived from Events + time + GPS + external feeds + user overrides

How they interact

The single most important architectural decision in this spec: **Event is the universal timeline unit, and everything else attaches to it.** A Journey, a hotel stay, a temple visit, a meal, and a free-time block are all just Events with a different **type** and different type-specific detail. The Home screen, Timeline, and state engine never branch on *what kind of trip this is* — only on `event.type` and `location.category`, which are data fields, never hardcoded strings tied to

a real place.

```

Trip
+- Travelers[]
+- Settings
+- Events[] (ordered, time-spanning)
    +- type: journey      -> Segments[] (1 or more legs)
    +- type: stay         -> check-in / check-out pair
    +- type: visit        -> Activities[]
    +- type: free_time    -> (no activities required)
    +- type: custom       -> generic fallback
Every Event may also have:
    +- Location reference(s)
    +- Task(s)
    +- Media[] (user photos/notes)

```

A Location is a reusable place record (coordinates, category, name) that many Events can point to. Photo Inspiration and default Activities are attached to the *category* of a Location (**temple**, **palace**, **lake**, **station**, **hotel**...), never to a named place — that is what makes a temple in Udaipur and a temple in Chennai behave identically without either being special-cased.

Current Trip State is deliberately **not persisted as its own table**. It is a runtime computation, described fully in Section 5.

4. Complete Trip Flow

This section narrates the Udaipur trip as a *sequence of Events*. Every row below is one Event record — not a hand-built screen. A different trip produces a different sequence of the same handful of Event types; no new screen is ever required to support it.

#	Event type	Udaipur-trip	example	Location(s)	Notes
1	Journey	Nagpur -> Agra (train)	Nagpur station -> Agra Cantt	One or more Segments	
2	Free time	Arrival buffer in Agra	Agra Cantt area	Short, unplanned-length gap	

#	Event type	Location(s)	Notes
Udaipur-trip example			
3	Visit	Agra layover — Taj Mahal	Taj Mahal Contains Activities: entry, viewing, photos, exit
4	Visit (optional)	Other Agra sight or meal	Agra Same generic Visit type
5	Journey (train)	Agra -> Udaipur	Agra Cantt -> Udaipur station Second long-haul leg
6	Stay	Hotel check-in	Hotel in Udaipur stay event spans multiple days
7	Visit (repeated)	City Palace / Lake Pichola / Jagdish Temple / etc.	Udaipur POIs Each is its own Visit Event with its own Activities
8	Free time	Between visits	— Data-driven gap, not scripted
9	Stay	Hotel check-out	Hotel in Udaipur Closes the Stay event
10	Journey (train)	Udaipur -> Nagpur (return)	Udaipur station -> Nagpur station Mirrors Event 1

The engine has no concept of “Agra layover” or “Taj Mahal” as special cases — it only ever reasons about “Event #3, type=visit, Location X, window [A,B], N Activities, status=upcoming.” The exact same rendering, state logic, and navigation logic apply whether Location X is the Taj Mahal or a beach in Goa.

Opinion: treat check-in and check-out as the *boundaries* of a single **stay** Event, not two separate Events — a hotel is one continuous state (“where you’re based”), and modeling it as one Event with two timestamps avoids the Timeline showing a hotel “ending” and “starting” again with nothing between.

5. Current Trip State

This is the engine that answers “what’s happening right now.” It runs on a timer (e.g., every 30–60 seconds) and on every meaningful GPS update, taking these inputs:

- Current timestamp
- Current GPS position (if permitted and available), with an accuracy radius
- The Trip’s ordered Event list, each with planned times, current status, and Location geofence
- External live-status feeds where available (train running status)
- Any manual user overrides already applied

...and produces this output: the **current Event** (or none), the **next Event**, a **trip-level state label**, and any derived alerts (delayed, leave-now, schedule conflict).

State definitions

State	Meaning	Applies to	Typical trigger
Upcoming	Not yet started, outside the pre-start window	Any Event	Default before start
Starting	Inside the pre-start	Any Event	$\text{now} \geq \text{start} - \text{threshold}$
Soon	threshold	Event	now between departure and arrival
In Transit	Currently moving between Locations	Journey / Segment	
Approaching	Nearing the destination	Journey, or travel toward a Visit	GPS inside outer geofence, or live feed/ETA says close
Arrived	At the destination, not yet marked active	Journey / Visit	GPS inside inner geofence
Active	This is the current Event	Any Event	In time window AND/OR arrived AND/OR user-confirmed
Completed	Finished	Any Event	End time passed + next started, or user confirms, or geofence exited and stayed exited

State	Meaning	Applies to	Typical trigger
Delayed	Behind schedule	Mostly Journeys, but any Event	Elapsed time or live feed exceeds plan by a threshold
Missed	Window fully passed with no Active/Completed/Skipped	Any Event	<code>now > end</code> , never activated
Skipped	User explicitly bypassed it	Any Event	Manual override only
Free Time	No scheduled Event is current; a planned gap block	Trip-level, or <code>free_time</code> Event	Between two Events, matches a planned gap
Waiting for Next Event	Current Event completed, next hasn't started, and no Free Time block was planned for this gap	Trip-level	Short, unplanned gap

Deviation handling

The engine runs at one of three **confidence tiers**, degrading gracefully:

1. **GPS + time (high confidence)**: full automation, geofence-driven transitions.
2. **Time only (medium confidence)**: GPS denied or unavailable — the engine relies on the schedule and waits for manual “I’m here” confirmations for anything GPS would normally decide.
3. **Stale / offline (low confidence)**: plan-only mode. Live feeds disabled, everything schedule-driven, clearly labeled as such.

When reality diverges from plan — a Visit runs long, a train is late — the engine **shifts only the estimated times of downstream flexible Events**, never the fixed, real-world-committed ones (a booked train’s departure does not move because the user is running late). This distinction is a field on every Event: `timingType: fixed | estimated`.

The engine never silently rewrites the schedule without telling the user. A visible, low-key “Running ~25 min behind” indicator appears on Home whenever the observed delta exceeds a small threshold. If a deviation would create a **hard conflict** (e.g., a shifted arrival now lands after a fixed-departure train), the engine escalates to a “Schedule conflict” banner and asks the user to skip,

reorder, or edit — it never guesses silently.

6. Home / Command Center

The Home screen exists to answer eight questions with zero taps. Each is mapped to a specific element:

Question Answered by	
Where am I?	Right Now hero card + mini map
What am I doing now?	Current Event card
What's next?	Next Up card
Where do I need to go?	Next Up card's Location name + mini map pin
How far away is it?	Distance readout on Next Up card
How long will it take?	ETA readout on Next Up card
When should I leave?	"Leave by" callout, or a Leave-Now alert
What should I do when I arrive?	Tapping through to the Location Detail / Activities list

Cards, top to bottom

1. **Trip identity bar** — trip name, "Day X of N," a trip switcher if the user has more than one trip stored.
2. **Right Now hero** — the current trip-state label in large type ("In Transit," "At City Palace," "Free Time"), with the current or nearest Location name underneath.
3. **Current Event card** — shown only when an Event is Active: name, time window, a progress indicator (elapsed vs. planned duration), and the primary action for that Event type (e.g., "Mark Arrived," "View Activities," "Mark Complete").
4. **Next Up card** — next Event's name and Location, scheduled time, live distance and ETA from the user's current position, a "Leave by HH:MM" callout, and a prominent **Navigate** button.
5. **Mini timeline strip** — a horizontal, scrollable preview of today's remaining Events with small status dots; tapping opens the full Timeline scrolled to "now."
6. **Mini map** — a small live snippet with the user's pin and the next destination's pin; tapping expands to the full Map tab.
7. **Quick actions row** — Navigate, Mark Complete, Add Photo/Note, View Timeline.
8. **Alerts area** — delay warnings, schedule-conflict banners, offline indicator. Empty and invisible when there's nothing to say — this area must never show a placeholder just to fill space.

9. **Free Time card** — replaces the Current Event card during Free Time: a light suggestion (e.g., nearby Photo Inspiration, “add something?”) rather than an empty state.
10. **Weather snippet** — small, secondary, current-location weather. Not MVP-critical.

Opinion: the Next Up card, not the Current Event card, is the single most important element on the screen — the moment of highest anxiety in real travel is “am I going to make this,” not “what am I doing this second.” Design review should protect its prominence above everything else on Home.

7. Timeline

A single vertical, chronological list, grouped by day (tabs or sticky day headers for multi-day trips). Every Event — train, hotel, temple, restaurant, free-time block — renders through **the same node component**, differentiated only by data:

- **Icon** — resolved from `event.type` (and `location.category` for Visits) via an icon lookup table, never a per-place special case.
- **Time range, title, Location name.**
- **Status pill/color** — matching the state table in Section 5 (e.g., muted gray = Upcoming, highlighted/pulsing = Active, faded check = Completed, amber = Delayed, red outline = Missed).
- **Subtitle** — Activity count for Visits, distance for Journeys, or nothing for Free Time.

A live “**now**” **marker** sits at the current time position and the Timeline auto-scrolls to it on open. Tapping a node expands it inline (Activities checklist, notes, photo count) without navigating away; a secondary tap opens the full Location Detail page. Past, completed Events auto-collapse to a one-line summary to keep the scroll length manageable, but remain expandable. Filter chips (All / Transit / Visits / Free time) are a nice-to-have, not MVP.

8. Map and Location Tracking

- **Live user location** — a standard “blue dot” pin with an accuracy ring.
- **Planned locations** — pins for the current day’s (and nearby days’) Events; the current destination pin is visually emphasized (larger, distinct color) over the rest.
- **Route to next destination** — a polyline from a routing provider, with distance and ETA overlaid near the destination pin.
- **Arrival detection (geofencing)** — every Location has a configurable arrival radius, with sensible category defaults (e.g., ~150m for an attrac-

tion, ~300m for a hotel, ~500m for a station/large area) and an explicit per-Location override for cases that need it.

- **GPS inaccuracy handling** — fixes with an accuracy radius worse than a threshold (e.g., >100m) are ignored for state-changing decisions; the engine requires several consecutive good fixes before flipping a geofence-based state, to prevent flapping from a single noisy reading.
- **Location permissions** — the app must remain functional in a degraded, time-only mode if permission is denied; a persistent-but-unobtrusive prompt offers re-enabling, never a blocking modal.
- **GPS unavailable** — show a small “Location unavailable — using schedule” banner and fall back entirely to the time-only confidence tier (Section 5), with a manual “I’m here” button as an override.
- **Dynamic focus** — the map auto-fits to bound the user’s position and the next one or two relevant Locations. This re-fit happens on major state transitions, not continuously (to avoid jitter), and a floating “recenter” control restores auto-focus after the user pans away.

9. Navigation

Tapping **Navigate**:

1. Resolves the target Location — the current Event’s Location if In Transit doesn’t apply, otherwise the next Event’s Location.
2. Presents a lightweight choice: hand off to the device’s default maps app for turn-by-turn (the pragmatic MVP default — building full in-app turn-by-turn is out of scope), or show a simple in-app route line for a quick glance.
3. Logs a `navigationStarted` timestamp, which the state engine uses to lean toward In Transit even slightly before GPS confirms movement.
4. On return to the app, the engine re-evaluates state from a fresh GPS read — nothing needs to be manually “resumed.”

Offline fallback: show straight-line distance and, if available, a last-cached route polyline; live ETA is disabled and marked as such.

10. Train Journey Experience

A train Journey is a Segment carrying this data:

- Origin and destination station/Location
- Scheduled departure and arrival
- Carrier/service identifier (train name/number) — data, not hardcoded
- Live running status if a feed is available (on time / delayed by X minutes, and current running station where the feed supports it)

- Platform, if known/updated
- Coach/seat reference (optional, informational)
- Distance remaining and a recalculated ETA when live data is present

Status flow: Upcoming -> Starting Soon (as departure nears) -> In Transit (departure time reached, or the live feed confirms movement) -> Approaching (live feed nears the destination station, or scheduled arrival minus a buffer is reached with no live feed) -> Arrived (feed confirms arrival, or scheduled time + buffer passes) -> Completed, which automatically activates the next Event (e.g., a layover Visit or a Stay check-in) after a short grace window.

If no live feed is available for a given train service, the entire flow degrades gracefully to the time-only path — the UI must not show a “live” indicator it cannot back up.

11. Photo Inspiration and Planned Shots

Photography is a first-class destination feature, but it must remain lightweight and useful. The application should distinguish between **inspiration** and **planned shots**.

Inspiration

For any Location, the app may show a small set of relevant photography examples or links based on the Location name, category, tags, and optionally time-of-day. This can begin with a simple category-based provider and later use external sources.

Examples of inspiration categories:

- Portrait
- Couple
- Architecture
- Landscape
- Sunset / golden hour
- Candid
- Food
- Street
- Reflection
- Framing / composition

The provider must remain pluggable, but **no LLM is required**.

Planned shots

A user can select an inspiration and turn it into a concrete shot they intend to capture. A Planned Shot contains:

- Title
- Reference image or link (optional)
- Pose / composition description
- Notes
- Status: planned / taken / skipped

This creates a small, actionable shot list rather than a passive gallery.

Destination photography flowchart

```

Location
|
Category + name + tags
|
Photo inspiration
|
Select idea(s)
|
Planned Shot list
|
During visit
|
Mark as Taken / Skip

```

The current MVP should not depend on scraping Pinterest. Pinterest or other sources can be integrated later behind the same provider interface.

12. Location Detail Page

The full screen for a single destination (City Palace, Lake Pichola, Jagdish Temple, or any other Location):

- Hero image (from Location media, or the user’s own first photo once captured)
- Name, category chip, one-line description
- Live distance and ETA from the user’s current position, with a Navigate CTA
- Scheduled visit window (editable)
- Activities checklist (Section 13)
- Photo Inspiration carousel (Section 11)
- Planned Shots list with quick status changes
- Notes field — tied to *this trip’s visit* to the Location (an event-instance note), auto-saving free text
- Completion status, with a manual “Mark Visited” override always available
- Nearby Places — other Locations (itinerary or discoverable) within a radius; secondary, depends on a places-search capability
- User photo gallery for this visit

- Optional checklist (things to bring/remember at this stop) — can reuse the generic Task entity scoped to this Event rather than inventing a separate system

13. Activity System

An Activity is a specific thing to do inside a Visit Event: `id`, `parent eventId`, `title`, `type/category`, `order`, `isOptional`, `estimatedMinutes` (optional), `completionStatus`, `notes`. Activities render as an ordered, tappable checklist under the relevant Event.

Example: **Location -> City Palace**, **Event -> Visit** contains Activities: explore museum, visit courtyard, architecture photos, couple photos, leave location. Completing every Activity is informational, not gating — it never forces the parent Event into Completed automatically (see Section 14’s reasoning for why Visit completion needs its own confirmation).

Recommendation — Activity templates: a Location’s *category* can carry a default suggested Activity list (every **palace**-category Location defaults to something like “Explore grounds / Photos / Museum if applicable”). When the Trip Editor creates a Visit Event against that Location, the template is copied in as a starting point, editable per instance. This removes repetitive manual entry without ever hardcoding content to a named place.

14. Automatic Progression

Transition	Trigger	Mode
Upcoming -> Starting	Time threshold reached	Automatic
Soon Starting Soon -> In Transit (Journeys)	Departure time reached, or user starts navigation	Automatic
In Transit -> Approaching	GPS in outer geofence, or live feed/ETA under threshold	Automatic if a signal exists; else time-based fallback
Approaching -> Arrived	GPS in inner geofence	Automatic, with manual “I’m here” fallback

Transition Trigger Mode		
Arrived -> Active (Visit/Stay)	Immediate or short grace period	Automatic
Active -> Completed (Journeys)	Arrival geofence/feed confirmed + short buffer	Automatic
Active -> Completed (Visits/Activities)	User confirms, or next Event's Starting-Soon fires, or GPS shows sustained departure from the geofence	Confirmation preferred — the engine may prompt, but should not silently auto-complete a Visit
Any -> Delayed	Elapsed time or live feed exceeds plan by threshold	Automatic flag, doesn't change base status
Any -> Skipped	—	Manual only
Any -> Missed	Window fully elapsed, never activated	Automatic flag; late manual completion still allowed

Opinion, and the reasoning behind it: transit arrival is well-defined and safe to fully automate. How long a person actually spends enjoying a palace is not — auto-completing a Visit the instant a scheduled window ends would frequently be wrong and would erode trust in the whole system. Prefer a gentle prompt (“Wrapping up at City Palace?”) over silent automation for anything experience-based.

15. User Overrides

- **Skip** — marks the Event Skipped; downstream estimated times re-baseline; the user is notified only if it changes the next Leave-by time.
- **Complete early** — marks Completed now; the next Event is immediately re-evaluated (may jump straight to Active if already inside its window).
- **Delay** — edit the time fields directly, or a quick “Push back 30 min” action; only Events with `timingType: estimated` shift automatically — fixed, real-world-committed Events (a booked train) never move because the user is late.
- **Reorder** — drag-and-drop in the Trip Editor or a Timeline edit mode; only valid between Events without fixed timing constraints.
- **Add a new place or spontaneous activity** — a quick-add affordance from a long-press on the Map, a “+ Insert stop” on the Timeline, or directly from the Free Time card; creates a new Event inline, inheriting the Trip’s travelers and settings.

- **Change timings** — a direct edit sheet on any Event.
- **Manual completion** — available on every Event regardless of computed state, always.
- **Pause tracking** — a global toggle that stops GPS polling and automatic transitions; state freezes at its last computed value and re-evaluates fresh the moment tracking resumes. Useful for battery, privacy, or when automation has clearly gotten something wrong.

The application must remain fully functional the moment the real trip diverges from the plan — none of the above are optional escape hatches bolted on afterward; they are core to the product being trustworthy.

16. Plug-and-Play Trip Creation

The Trip Editor must optimize for a very simple real-world workflow:

Select location -> enter time -> add activities -> optionally add photo plans -> save.

The user must not need to understand the underlying data model.

Add location from map

A map is a first-class location entry tool. The user can open **Add Location**, pan/zoom the map, and select a point. The application resolves the selected coordinates to a human-readable place name/address where possible.

```
+ Add Location
  |
  Open Map
  |
  Move / zoom map
  |
  Drop / confirm point
  |
  Resolve name + address
  |
  Set date + time
  |
  Add activities
  |
  Add planned photo shots (optional)
  |
  Save
```

Optional location search

Map selection is primary, but a search/autocomplete field may be offered for convenience. The final saved Location must store coordinates as the source of truth.

Optional Google Maps link input

A future convenience feature may accept a Google Maps link and resolve it into a Location, but this must not be required for the core workflow.

Quick Add

The user should be able to add a spontaneous stop while travelling without leaving the current context. A quick-add action should open a compact version of the same workflow.

17. Trip Editor

The Editor is what proves the architecture is generic: it must be able to build a trip that has nothing to do with Udaipur, using the same forms.

Event type		
Required fields		
Optional fields		
Train / Flight / Bus (Journey)	origin Location, destination Location, departure time, arrival time, mode	carrier/service name, service number, platform/gate, seat/coach, booking reference
Car / self-drive (Journey)	origin, destination, estimated departure	route notes
Hotel (Stay)	Location, check-in date/time, check-out date/time	booking reference, room type, notes
Place / attraction (Visit)	Location, planned date, planned time window	category tag, Activities list, notes
Restaurant (Visit/meal)	Location, planned time	cuisine tag, reservation info
Standalone activity	title, time (or “anytime” flag)	linked Location, notes
Free-time block	date, time window	suggestion tag (e.g., “near hotel”)
Custom event	title, date/time	Location, notes, icon choice

A Trip is created with a name, date range, and travelers, then populated

with Events of any of the above types, in any order, reorderable at any time. **Duplicate-as-template** (copy an existing trip’s structure, then bulk-edit the Locations to a new city) is a strong, cheap secondary feature — and doubles as the best possible acceptance test for genericity: if duplicating the Udaipur trip and swapping in Goa’s Locations works with zero code changes, the architecture is correct.

18. Data Model

Conceptual schema only — no implementation code, translatable directly into MongoDB or any other store.

Trip

Field	Type	Notes
id	id	
name	string	
startDate / endDate	date	
status	enum	planned / active / completed / archived
travelerIds	id[]	
settings	object	units, notification prefs

Event (universal fields — every Event has these, regardless of type)

Field	Type	Notes
id	id	
tripId	id	
type	enum	journey / stay / visit / free_time / custom
title	string	
startTime / endTime	datetime	
timingType	enum	fixed / estimated — governs whether overrides can shift it
status	enum	matches Section 5’s state list
order	number	position in the trip sequence
primaryLocationId	id	
typeDetailsId	id	points to the matching detail table below

Type-specific detail tables (one-to-one with an Event, keyed by **type**) — kept *separate* from Event on purpose, so Event itself never accumulates a pile of nullable, type-specific columns:

- **JourneyDetails:** originLocationId, destinationLocationId, mode, carrier, serviceNumber, platform, segments[]
- **StayDetails:** locationId, checkIn, checkOut, bookingRef
- **VisitDetails:** locationId, activityIds[]
- **FreeTimeDetails:** suggestionTag (optional)
- **CustomDetails:** notes, iconKey

Location

Field Type	
Notes	
id	id
name, address	string
coordinates	geo point
category	enum palace, lake, temple, station, hotel, restaurant, viewpoint, custom...
geofenceRadius	number defaults by category, overridable

Activity — id, eventId, title, category, order, isOptional, estimatedMinutes, completionStatus, notes. **Task** — id, tripId, eventId (optional), title, dueAt (optional), isDone. **Media** — id, tripId, eventId (optional), locationId (optional), type (photo/note), uri or text, capturedAt. **Traveler** — id, name, role.

Data / Business Logic / UI — explicit separation

- **DATA:** everything above. This is the only layer where the word “Udaipur” or a specific set of coordinates is ever allowed to appear — and only as *row values*, entered through the Trip Editor or an import, never as a literal in code.
- **BUSINESS LOGIC:** the state engine (Section 5), geofencing math, progression rules (Section 14), notification triggers (Section 20), re-baselining. This layer reads `event.type` and `location.category` — never a place’s name. Wrong: `if (location.name === "City Palace") { showPalaceActivities() }`. Right: activities render from `event.activities`, whatever they contain.
- **UI:** screens and components that render whatever the logic layer hands them. Copy is templated with variables — “Time to leave for {**event-Name**},” never “Time to leave for City Palace” baked into a string resource.

This is the architectural decision that most determines whether the app is generic or a disguised one-trip app. If a reviewer can grep the codebase for “Udaipur,” “Agra,” “City Palace,” or “Nagpur” and find anything outside of seed/sample data files, that is a defect against this spec.

19. External Services

Describe the *capability* needed; no specific vendor is assumed except where the platform forces one.

- **Maps & tile rendering** — display pins, polylines.
- **Geocoding** — forward (address -> coordinates) and reverse (coordinates -> address), mainly for the Trip Editor.
- **Routing/directions** — distance, ETA, multi-mode (walking, driving, transit).
- **Places search/autocomplete** — convenience when adding a new Location in the Editor. Secondary.
- **Live transit status** — given a service identifier and date, return current status/ETA/delay; naturally region-specific (India's rail network vs. flight-status APIs elsewhere), so this must sit behind one abstract interface with swappable implementations.
- **Photo inspiration provider** — as defined in Section 11.
- **Weather** — current + short forecast by coordinates. Secondary.
- **Push notifications** — device-level delivery (platform push service); everything else can be abstracted, this one is inherently platform-specific.
- **GPS/location** — OS-level positioning; also inherently platform-specific and not abstracted away like the others.

Everything except raw GPS and push delivery should sit behind an internal interface so a concrete provider can be swapped without touching business logic or UI.

20. Notifications

Notification Trigger	
Leave now	<code>scheduled start - travel time - buffer</code> reached
Event starting soon	Time threshold before a non-transit Event
Arriving soon	GPS/live feed enters the outer geofence
You've arrived	GPS/live feed enters the inner geofence
Train delayed	Live feed reports a delay beyond threshold
Next destination reminder	An Event completes and the next hasn't auto-started after a short window
Activity reminder	Optional, low-priority nudge about an unfinished Activity mid-visit

Anti-spam rules: cap notifications per Event; apply hysteresis around geofence boundaries so GPS jitter near an edge cannot fire “arrived” / “left”

repeatedly; make Activity reminders opt-in and off by default.

21. Offline Behavior

- The entire itinerary (Events, Locations, Activities) is cached locally on load/sync, so the core schedule and Timeline work with **zero network**.
- GPS itself needs no network to produce a position fix; only reverse-geocoding and live map tiles do, and those degrade to cached/last-known versions.
- Distance/ETA falls back to a straight-line (haversine) estimate when the routing provider is unreachable, clearly labeled as an estimate.
- Live feeds (train status, weather) degrade to “last known, as of [time]” rather than disappearing or erroring.
- Photo Inspiration falls back to cached items or hides gracefully.
- Notifications that depend only on local time/GPS (Leave-now, Arriving-soon) still fire offline; ones dependent on external feeds (delay alerts) simply wait for connectivity.
- User actions (overrides, completions, photos) queue locally and sync once back online.

Principle: the trip’s core itinerary is never network-dependent to be *usable* — only to be *live-enhanced*.

22. Photo and Memory System

The user can attach Media (a photo, a short note) to any Event or Location instance; timestamp and location are captured automatically. This is deliberately simple for MVP — a photo/note attached to a point in the trip — and becomes the raw material for the End-of-Trip recap (Section 23) without needing its own separate system.

23. End-of-Trip Experience

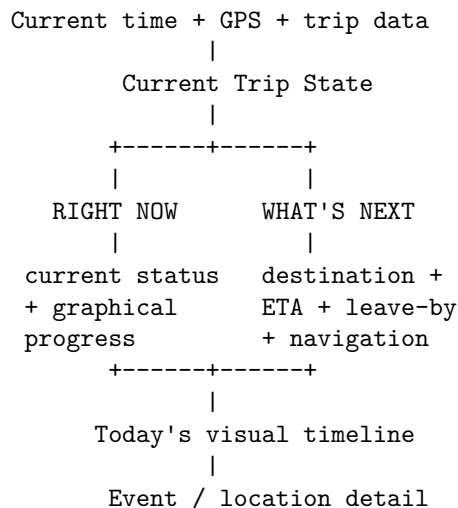
A Trip moves to **completed** once its last Event’s window has passed (or the user marks it complete). The recap screen surfaces:

- Locations visited vs. planned, and total distance traveled
- A condensed Timeline recap
- Captured photos, grouped by Location/day
- Any notes left along the way
- Simple trip statistics (days, stops, photos)
- Optional shareable summary card

24. UI/UX Principles

- **Mobile-first, one-handed:** primary actions reachable by thumb.
- **Glanceable over dense:** large status/progress elements, restrained metadata, and minimal text.
- **Current-vs-next hierarchy:** the current state and the next required action remain visually dominant.
- **Graphical communication:** prefer progress lines, rings, route graphics, timeline markers, distance visuals, icons, and large numeric displays over long explanatory blocks.
- **Pretty without clutter:** visual richness should come from layout, typography, imagery, motion, and graphical indicators rather than more cards.
- **Maps as a first-class interaction:** location selection and current position should feel native to the app.
- **Photography as an accent:** destination images and planned shots should enrich the experience without turning every screen into a gallery.
- **Native Android behavior:** use Android interaction patterns where they improve usability rather than forcing web-style interactions into the app.
- **Honest state communication:** never imply live/automatic confidence that the engine does not have.
- **Theme consistency:** both supported themes must preserve identical information hierarchy and behavior.

Primary Home / Trip HUD flowchart



The Home screen should feel like a **visual trip HUD**, not a generic dashboard.

Recommended visual components

- Graphical trip progress line
- Circular or linear event progress
- Route-to-next-destination indicator
- Large current time
- Large distance / ETA values
- Compact day progress indicator
- Timeline markers
- Destination photography
- Minimal status badges

These are visual components, not separate data systems. Their values come from the same Trip Engine state.

25. Visual Design System

The app must support **two complete visual themes**. Themes change presentation only; they must never change trip logic, stored data, state calculations, or feature availability.

Theme A - Nothing OS inspired

This is an original interface inspired by the visual qualities of Nothing OS, not a direct reproduction of proprietary assets.

- Black, white, and restrained grayscale foundation
- High use of negative space
- Thin rules and technical dividers
- Monospaced / technical typography for data and timestamps where appropriate
- Dot, segmented, circular, and line-based visual indicators
- Large numbers for time, distance, progress, and status
- Minimal but deliberate animation
- Quiet surfaces with strong hierarchy

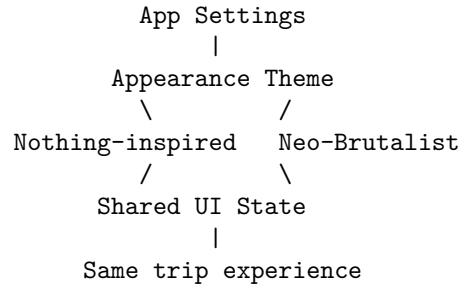
Theme B - Neo-Brutalism

- Yellow + black as the primary identity
- Thick borders
- Hard offset shadows
- High-contrast typography
- Chunky cards and controls
- Strong rectangular geometry
- Deliberately visible structure rather than soft cards
- Limited gradients and visual decoration

Theme rules

- The same screen, component, data, and behavior must exist in both themes.
- Theme switching must happen without rewriting screen logic.
- The UI should be **pretty and visually distinctive without being visually heavy**.
- Use visual encoding instead of adding more text.
- Do not turn every screen into a dense dashboard.
- Map/navigation screens should prioritize clarity over decorative styling.
- The visual design should use photography and graphical indicators as accents, not as clutter.

Theme switching flowchart



The final typography, spacing scale, border weights, shadows, corner treatment, icon language, and motion system should be defined after reviewing a few concrete visual references. The implementation should keep these tokens centralized so the design can evolve without changing product logic.

26. Edge Cases

Scenario	
Expected behavior	
User is late	Current Event stays Active past its window; Delayed flag appears; downstream <i>estimated</i> Events shift; fixed Events don't
User arrives early	Approaching/Arrived fires early via GPS; Active starts early too, no penalty
User skips a destination	Manual Skip; downstream re-baselines; no Missed flag applied
User is far from planned route	Engine lowers confidence, may surface "off planned route" rather than guessing a state; never silently reassigns a different Event as current

Scenario Expected behavior	
GPS jumps unexpectedly	Requires consecutive consistent fixes before acting; single-fix jumps are ignored
Train is delayed	Live feed updates JourneyDetails; downstream estimated Events shift; Delayed flag shown with source (“per live status”)
Event overlaps another	Flagged as a Schedule Conflict at edit time and at runtime; user resolves via edit/skip/reorder
User manually changes the schedule	Treated as authoritative; engine re-baselines around it, no “correction” back to original plan
A location is closed/unavailable	User can Skip or edit the Event directly — the app doesn’t independently know opening-hours status without an added external check, which is a secondary capability, not MVP
Internet disappears	Falls back per Section 21; itinerary remains fully browsable
An API fails	That specific capability degrades (e.g., no live train status) without affecting unrelated features; failures are isolated per-service
User adds an unexpected destination	Quick-add creates a new Event inline (Section 16); fully supported without leaving the current view

27. Genericity Requirement

Every naive shortcut that would tie the app to this one trip, replaced with the generic mechanism actually required:

Hardcoding trap instead	Generic mechanism
UI copy like “Welcome to your Udaipur trip”	Templated copy pulling <code>trip.name</code>
Assuming exactly three cities/stops	Trips hold an arbitrarily long, arbitrarily ordered Event list
Assuming train is the only transport mode	<code>mode</code> is a field on Journey; UI renders per-mode icon/detail sets from a lookup table

Hardcoding trap instead	Generic mechanism
-------------------------	-------------------

Coordinates for City Palace, Lake Pichola, etc. baked into code	Coordinates live only in Location rows, entered via the Editor or import
A single traveler/couple assumed	Traveler list of arbitrary size and role
A fixed geofence radius tuned for one monument	Radius defaults by Location <i>category</i> , overridable per Location
A hand-written photo-inspiration list for named places	Pluggable Inspiration Provider keyed by category/tags (Section 11)
A hardcoded “3-city” progress indicator	Progress computed from the live count/order of Events
Activities hand-written per named location	Activities are Event-owned data, optionally seeded from a category template (Section 13)
A schema field bolted on per event type (e.g., <code>trainNumber</code> sitting directly on Event)	Type-specific detail lives in separate *Details tables (Section 18)

Acceptance test for the whole architecture: duplicate the Udaipur trip in the Editor, bulk-replace its Locations with a different city’s, and confirm every screen renders correctly with zero code changes. If that test fails anywhere, the failure point is exactly where genericity broke down.

28. Expected User Experience

Waking up (Day 1, Nagpur): the Home screen already shows the day’s shape — a train departure this morning, a first-time confidence tag (“schedule-based” until GPS engages), and a Leave-by time for reaching the station.

On the train: the Current Event card shows the Journey in progress, elapsed vs. remaining time, and — if a live feed exists — a running-status line. The map focuses loosely on the route rather than a precise pin, since a moving train doesn’t need second-by-second tracking to feel useful.

Reaching Agra: as the train nears the station, state moves through Approaching -> Arrived automatically; the app immediately promotes the next Event (the Agra layover) and shows the Leave-by/Navigate prompt for it without the user having to do anything.

During the layover: the Location Detail page for the layover stop shows the Activities checklist and Photo Inspiration for that category of place, distance already resolved from the current position — no searching required.

Traveling to Udaipur: identical experience to the Nagpur–Agra leg — same card, same states, same interactions, because it is the same Event type with

different data.

Arriving at a destination (e.g., City Palace): geofence entry flips state to Arrived/Active; the Home hero updates instantly; the Activities list is right there, already populated.

When an activity ends: rather than silently auto-completing, the app gently asks “Wrapping up here?” — one tap either confirms or extends.

Free time: the Current Event card is replaced by a soft suggestion (nearby inspiration, or simply “next stop is at HH:MM”) — never a blank, confusing gap.

End of trip: the last Event completes, and the app offers the recap — what was seen, how far traveled, the photos taken along the way — closing the loop the same way it opened it, without the user ever having had to consult a separate document.

Appendix A. MVP Scope & Build Order

The MVP is deliberately narrow. It must first prove that a generic, data-driven trip engine works as a polished native Android experience.

Tier 0 - Native Android foundation

- Kotlin + Jetpack Compose project
- Navigation
- Room persistence
- DataStore settings
- ViewModel / StateFlow structure
- Theme tokens and both visual themes

Acceptance test: switching between themes does not change any trip behavior or data.

Tier 1 - Plug-and-play trip creation

- Create trip
- Add Location from map
- Set date/time
- Add activities
- Add optional planned shots
- Edit/reorder/delete events
- Save locally

Acceptance test: create two unrelated trips without changing code.

Tier 2 - Core trip HUD

- Home / Trip HUD
- Current event
- Next event
- Visual trip progress
- Timeline
- Location detail
- Manual complete / skip / edit
- Map tab

Acceptance test: the UI remains useful with GPS completely disabled.

Tier 3 - Location automation

- Native GPS integration
- Arrival geofencing
- Automatic current/next state updates
- Leave-now calculation
- Android notifications
- Travel Mode tracking

Acceptance test: a simulated trip can move through events using time + location without manually changing each state.

Tier 4 - Live transport and enrichment

- Live train status provider
- Photo inspiration provider
- Planned shot workflow improvements
- Offline caching improvements
- Weather

These features must remain optional. The core application must work without them.

Explicitly out of scope for the current version

- LLM controlling the itinerary
- LLM automatically rewriting the schedule
- AI deciding what event should happen next
- AI-generated navigation decisions
- AI-dependent core functionality
- Full in-app turn-by-turn navigation
- Social travel network features
- Full booking engine

The application may experiment with AI in the future as a separate, optional layer, but the current product must remain fully functional without an LLM.

Appendix B. Final Architecture Principles

The implementation should pass the following rules before being considered complete:

1. **Trip data is configuration, not code.**
2. **Event is the universal timeline unit.**
3. **Location coordinates are data entered through the editor, not hardcoded destinations.**
4. **The Trip Engine is deterministic.**
5. **GPS, time, and optional live feeds update state; they do not rewrite product logic.**
6. **Manual user overrides always exist.**
7. **The current MVP has no LLM dependency.**
8. **Nothing-inspired and Neo-Brutalist themes share the exact same data and behavior.**
9. **The map is a primary location-creation tool, not merely a display surface.**
10. **The Home screen communicates primarily through graphics, hierarchy, and concise text.**
11. **Photography is split into Inspiration and Planned Shots.**
12. **All core trip functionality remains usable offline.**

End-to-end product flowchart

